

ghactaint

Detecting and Repairing Expression Injection in GitHub Actions
IEEE Computer Society Global Student Challenge 2026 — Challenge 02 — Team 312

1. Summary

ghactaint is a fully deterministic static taint analyzer and automatic repair tool for expression-injection vulnerabilities in GitHub Actions. It makes zero LLM calls at runtime. Every number below is measured against the data on disk (150-sample labeled train split; validation for patch application), not estimated.

Task / metric	Result (measured on disk)
Detection - flow-level, exact from/to lines	TP 62 / FP 0 / FN 0 -> precision 1.000, recall 1.000, F1 1.000
Detection - sample-level	TP 31 / TN 119 / FP 0 / FN 0 -> accuracy 1.000
Patches apply (git apply --check) - train	31 / 31
Patches apply (git apply --check) - validation	16 / 16
Null baseline	F1 0.000 / accuracy 79.3%

The eval harness reports DIAGNOSIS: line convention consistent with gold on all matched flows — i.e. every gold flow found on the right file also landed on the exact right line.

2. Approach

A GitHub Actions expression is text-substituted into the runner's shell script before the shell parses it, so an attacker-controlled value (branch name, issue title, commit message) becomes shell source; quoting in the YAML does not help. The tool models this as interprocedural taint: sources are the predefined untrusted github.* contexts in untrusted_data.csv; sinks are run: scripts; taint propagates through with: inputs, GITHUB_ENV / GITHUB_OUTPUT / ::set-output writes, job and action outputs, and JavaScript/Docker actions (treated as opaque, endpoints stay YAML). uses: references resolve to the vendored actions/ and reusable_workflows/ under the split. The sink line is the line of the `${{ }}` interpolation; repair is env-hoisting (move the expression into a step-level env: entry and reference it as a quoted shell variable, so the value is delivered as data, not code).

3. Task 1 — Detection

On the 150 labeled train samples (31 vulnerable, 20.7%; 62 gold flows): flow-level TP 62 / FP 0 / FN 0 (precision, recall, F1 all 1.000) at exact from/to lines; sample-level TP 31 / TN 119 / FP 0 / FN 0 (accuracy 1.000). The null baseline (predict all clean) scores F1 0.000. Recall is the whole problem and no gold flow is missed; no clean sample is misflagged.

4. Task 2 — Repair

Repair is deterministic env-hoisting. Generated patches are single git-apply-compatible unified diffs, one file per vulnerable sample. All generated patches apply cleanly under git apply --check: 31/31 on train and 16/16 on validation. (Before the boolean-result refinement in section 6, validation reported 19 vulnerable samples / 28 flows and 19/19 patches applied; three samples became clean once their only flows were boolean-guarded false positives, giving the current 16/16.)

5. Four defects found and fixed

Each was found by running against the real dataset (not the synthetic fixture) and fixed with the root cause identified; all four are committed separately.

5.1 cp1252 encoding crash on non-Latin-1 characters

The verify step wrote the diff to a temporary file using the platform default codec (cp1252 on Windows). A diff containing a non-Latin-1 character — e.g. an emoji (U+1F680) present in a workflow — raised `UnicodeEncodeError` and aborted the whole run. Root cause: `open()` without an explicit encoding. Fix: write patch/temp files with `encoding=utf-8`.

5.2 Source misattribution: description prose vs default interpolation

For a composite-action input whose default self-taints (default: `${{ github.head_ref }}`), the source line was located by a textual scan of the context token, which matched an earlier description: prose mention (the string `'...{github.head_ref}'`) instead of the real default: interpolation — e.g. reported `action.yaml:9` (description) where gold is `:11` (default), breaking four flows. Root cause: first-textual-match of the token. Fix: take the source line from the default: value's own interpolation position. Train exact-line flow F1 went 0.928 -> 1.000.

5.3 CRLF write defect made every patch fail git apply

The `.patch` files (and verify temp files) were written in Windows text mode, translating each LF to CRLF. `git apply` then searched for CRLF-terminated context lines against the LF dataset files and every patch failed with 'patch does not apply', even though the patch content was correct (GNU patch, which strips CRs, applied them). Root cause: `open()` without `newline=""`. Fix: write with `newline=""`. Patches went from 0/31 to 31/31 on train and 0/19 to 19/19 on validation.

5.4 Whole-dataset copy per sample in verify (~50x speedup)

The self-verification step copied the entire ~160 MB dataset tree for every vulnerable sample (about 35 s warm / 210 s cold each; ~11 minutes for the validation set, which timed out once patches started applying). Root cause: a full copytree where only a few files are read. Fix: copy only the sample's read-closure (its workflow plus the transitive uses: closure that the analyzer actually reads), which is verdict-equivalent to the full copy and about 50x faster (full validation verify ~13 s). Equivalence was checked against the old full-copy path on samples spanning every sink type.

6. Boolean-result sink refinement (soundness)

Our sink criterion is whether an untrusted value can reach the shell as text. When an untrusted context is used only as an argument to a boolean predicate — `contains()`, `startsWith()`, `endsWith()` — the substituted value is the predicate's boolean result (true/false), or, in the guarded-literal form `startsWith(x, ...)` followed by hard-coded literal branches, one of a fixed set of literals. No attacker byte enters the rendered script, so the site cannot be an injection vector. A flow is suppressed only when every untrusted reference in the expression lies inside such a call (string literals are masked so their parentheses do not confuse the parse); if any reference escapes it — `raw`, or via `format()`, `join()`, or `toJSON()` — the flow is retained. This is a soundness-preserving refinement of the sink definition, not a scoring adjustment: it was verified inert on the labeled train set before shipping (train detection is unchanged at TP 62 / FP 0 / FN 0, because the boolean-only shape does not occur there).

7. Limitations (honest)

- One validation sample (63dd94ac) still produces malformed patched YAML: its patch fails verification gate 2 (YAML parse). Detection for it is unaffected; the patch shape needs further work.
- The boolean-result refinement has no train ground truth. The boolean-only shape does not occur in the labeled train set, so the refinement was verified inert there but its correctness on unseen data rests on the soundness argument (boolean output is not shell text), not on gold labels.
- Detection precision and recall are measured on the labels the model was refined against (train). The independent signal is that all generated patches apply cleanly under `git apply`.

- JavaScript and Docker actions are treated as opaque (any tainted input is assumed to reach every declared output). This is conservative: it cannot miss a flow but can over-approximate.

8. LLM and tool disclosure

The submitted pipeline makes zero LLM calls. No model is invoked during detection, repair, or verification; no API key is required; no model identifier applies. The competition rules require, if the system makes model calls, that they use OpenRouter with the organizer-issued team key and name the exact model — this system makes none. During development, an AI assistant (Anthropic Claude, via the Claude Code agent) was used as a programming aid: writing and refactoring code, analyzing the dataset, and drafting this document. Every empirical claim here was produced by running the tool and its eval harness on the data on disk.

9. Reproduction

Requires Python 3.9.7 with ruamel.yaml installed (pytest for tests; actionlint optional). Invoke the interpreter that has the dependencies — a bare python pointing at a different install without ruamel.yaml fails at import. Commands:

```
python -m pytest tests/ -q
python eval.py --self-test
python -m ghactaint.cli --root DATASET --split test --out . # -> test.csv +
patches/<sample_id>.patch
python -m ghactaint.cli --root DATASET --split train --out out_train
python eval.py --gold DATASET/train.csv --pred out_train/train.csv
```